

中山大学计算机院本科生实验报告

(2024年春季学期)

课程名称：并行程序设计

批改人：

实验	8-CUDA矩阵转置	专业	计算机科学与技术
学号	22336203	姓名	沈苇杭
Email	shenwh7@mail2.sysu.edu.cn	完成日期	2025.5.21

实验目的

- 掌握实验平台的使用方法
- 掌握CUDA编程的常见API
- 掌握使用CUDA并行解决问题的方法

实验过程与核心代码

HelloWorld程序

```
// Number of threads in each thread block
dim3 threadsPerBlock(m,k);

// Number of thread blocks in the grid
dim3 blocksInGrid = (n,1);
```

在CUDA中，线程块是程序的基本执行单元，由多个线程组成。在每个线程块中，线程可以共享一些资源。线程块可以是一维、二维或三维的结构。按照作业要求，这里的线程块是 $m * k$ 维度

线程网格是由一个或多个线程块组成的结构。按照作业要求，这里的网格是 $n * 1$ 维度。

```
hello_world_kernel<<<blocksInGrid, threadsPerBlock>>>(m,n,k)
```

在CUDA中，要调用一个核函数，需要传入：线程块数量，线程块中的线程数量、内核函数的参数。

```
cudaDeviceSynchronize();
```

并行结束前，需要等待所有线程完成任务。

```
cudaStatus = cudaDeviceReset();
if (cudaStatus != cudaSuccess) {
    fprintf(stderr, "cudaDeviceReset failed!");
    return 1;
}
```

程序运行结束前，需要重置当前的CUDA设备，释放其上的所有资源。

```
// CUDA kernel function
__global__ void hello_world_kernel(int m, int n, int k) {
    printf("Hello world from thread (%d, %d) in block %d", threadIdx.x,
        threadIdx.y, blockIdx.x);
}
```

在CUDA中，可以使用 `threadIdx` 和 `blockIdx` 获得线程索引和线程块索引。

矩阵转置

```
double *d_ptr = nullptr;
double *dt_ptr = nullptr;
err1 = cudaMalloc(&d_ptr, allocationSize);
err2 = cudaMalloc(&dt_ptr, allocationSize);
```

相比前一个实验，这个矩阵转置实验就会涉及到CUDA的内存管理了。在分配CPU内存的同时，也要使用 `cudaMalloc` 分配GPU内存。 `cudaMalloc` 的第一个参数是指向分配的GPU内存的指针，第二个参数是分配的内存大小。

```
cudaMemcpy(d_ptr, A, allocationSize, cudaMemcpyHostToDevice);
transpose<<<numBlocks, threadsPerBlock>>>(d_ptr, dt_ptr, n);
cudaMemcpy(AT, dt_ptr, allocationSize, cudaMemcpyDeviceToHost);
```

分配完内存后，我们还需要把CPU中的数据拷贝到GPU中，这时就要用到 `cudaMemcpy` 函数了。
`cudaMemcpy` 的参数是：目标内存的指针，源内存的指针，要复制的字节数，复制操作的方向。

```
dim3 threadsPerBlock(16,16);
dim3 numBlocks((n+threadsPerBlock.x-1) / (threadsPerBlock.x),
    (n+threadsPerBlock.y-1)/threadsPerBlock.y);
```

在这个实验中，同样需要设置线程块和网络的维度。

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);
cudaEventRecord(start);
cudaEventRecord(stop);
cudaEventsynchronize(stop);
float milliseconds = 0;
cudaEventElapsedTime(&milliseconds, start, stop);
```

在本实验中需要记录运行时间。

```
err1= cudaFree(d_ptr);
```

GPU内存需要使用 `cudaFree` 来释放。

实验结果

HelloWorld程序

```
ehpc@0a72bae315ce:~$ ./lab8-3 4 4 4
Hello World from thread (0, 0) in block 3
Hello World from thread (1, 0) in block 3
Hello World from thread (2, 0) in block 3
Hello World from thread (3, 0) in block 3
Hello World from thread (0, 1) in block 3
Hello World from thread (1, 1) in block 3
Hello World from thread (2, 1) in block 3
Hello World from thread (3, 1) in block 3
Hello World from thread (0, 2) in block 3
Hello World from thread (1, 2) in block 3
Hello World from thread (2, 2) in block 3
Hello World from thread (3, 2) in block 3
Hello World from thread (0, 3) in block 3
Hello World from thread (1, 3) in block 3
Hello World from thread (2, 3) in block 3
Hello World from thread (3, 3) in block 3
Hello World from thread (0, 0) in block 2
Hello World from thread (1, 0) in block 2
Hello World from thread (2, 0) in block 2
Hello World from thread (3, 0) in block 2
Hello World from thread (0, 1) in block 2
Hello World from thread (1, 1) in block 2
Hello World from thread (2, 1) in block 2
Hello World from thread (3, 1) in block 2
Hello World from thread (0, 2) in block 2
Hello World from thread (1, 2) in block 2
Hello World from thread (2, 2) in block 2
Hello World from thread (3, 2) in block 2
Hello World from thread (0, 3) in block 2
Hello World from thread (1, 3) in block 2
Hello World from thread (2, 3) in block 2
Hello World from thread (3, 3) in block 2
Hello World from thread (0, 0) in block 1
Hello World from thread (1, 0) in block 1
Hello World from thread (2, 0) in block 1
Hello World from thread (3, 0) in block 1
Hello World from thread (0, 1) in block 1
Hello World from thread (1, 1) in block 1
Hello World from thread (2, 1) in block 1
Hello World from thread (3, 1) in block 1
Hello World from thread (0, 2) in block 1
Hello World from thread (1, 2) in block 1
Hello World from thread (2, 2) in block 1
Hello World from thread (3, 2) in block 1
Hello World from thread (0, 3) in block 1
```

```
Hello World from thread (0, 3) in block 1
Hello World from thread (1, 3) in block 1
Hello World from thread (2, 3) in block 1
Hello World from thread (3, 3) in block 1
Hello World from thread (0, 0) in block 0
Hello World from thread (1, 0) in block 0
Hello World from thread (2, 0) in block 0
Hello World from thread (3, 0) in block 0
Hello World from thread (0, 1) in block 0
Hello World from thread (1, 1) in block 0
Hello World from thread (2, 1) in block 0
Hello World from thread (3, 1) in block 0
Hello World from thread (0, 2) in block 0
Hello World from thread (1, 2) in block 0
Hello World from thread (2, 2) in block 0
Hello World from thread (3, 2) in block 0
Hello World from thread (0, 3) in block 0
Hello World from thread (1, 3) in block 0
Hello World from thread (2, 3) in block 0
Hello World from thread (3, 3) in block 0
Hello world from the host!
ehpc@0a72bae315ce:~$
```

如图可见，线程输出的顺序有一定的规律。它会按照先输出一个线程块中的所有线程的结果，再输出下一个线程块的。最后，会输出主线程的结果。

矩阵转置

矩阵规模\线程块大小	8*8	16*16	32*32
512	2.46	3.29	2.43
1024	9.25	12.12	12.42
2048	36.13	37.32	48.10

上表记录了不同矩阵规模和线程块大小时的运行时间（单位：毫秒）。线程块大小相同的情况下，矩阵规模越大，运行时间越长。矩阵规模一定的情况下，随着线程块增大，运行时间可能减小、不变或增大。

此外，在实验中，还可以使用共享内存的方法来优化性能。使用共享内存，可以减少程序访问主存的次数，从而优化性能。

使用共享内存后，使用16*16的线程块和1024的矩阵规模，运行时间优化至：10.26s

实验感想

在本次实验中，我学会了使用基本的CUDA API实现并行的矩阵转置。在实验中，我遇到的主要问题是，不了解CUDA编程的API和流程。通过查阅博客，我了解了这些基本API的含义。